Chapter 1

RTL Design & Implementation of Digital Switch Debouncer

1.1 IP Block 6: Digital Switch Debouncer & Glitch Filter

In mixed-signal VLSI design, interfacing perfect digital logic with imperfect human-operated mechanics presents a severe reliability challenge. When a mechanical contact (such as a push-button or limit switch) closes, it does not transition instantaneously from 0V to 5V. Instead, the microscopic metal contacts physically collide, rebound, and vibrate. This phenomenon, known as **Contact Bounce**, generates a chaotic storm of high-frequency voltage spikes spanning hundreds of microseconds before the connection physically settles.

If this raw, bouncing analog signal is fed directly into a digital state machine or interrupt pin, a single physical button press will be interpreted as dozens of erratic clock cycles, causing catastrophic system failures. To eliminate this physical anomaly, a **Digital Switch Debouncer** was developed.

1.1.1 Detailed Working Mechanism & Theory of Operation

This IP block operates as a highly robust digital low-pass filter and signal conditioner, built upon two critical hardware stages:

1. Asynchronous Synchronization & Metastability Prevention

Because a physical switch is actuated independently of the FPGA's internal master clock, the bouncing input signal is strictly **asynchronous**. If a voltage spike crosses the digital logic threshold (V_{th}) at the exact moment the master clock transitions, it violates the critical **Setup Time** (T_{setup}) or **Hold Time** (T_{hold}) of the receiving flip-flop.

When these timing boundaries are violated, the flip-flop enters **Metastability**—a dangerous hardware state where the output voltage hovers unpredictably between logic 0 and logic 1, potentially propagating corrupted data throughout the entire system.

To prevent this, the raw input is routed through a **2-Stage Flip-Flop Synchronizer**.

- **Stage 1 (`sync_1`):** Captures the asynchronous input. If a setup/hold violation occurs, this register may become metastable.
- **Stage 2 (`sync_2`):** Provides a full clock cycle of physical resolution time, allowing the metastable voltage in Stage 1 to settle to a valid logic level before it is sampled by the rest of the system. This exponentially increases the Mean Time Between Failures (MTBF) of the circuit.

2. The Saturation Timer (Digital Glitch Rejection)

Once the signal is safely synchronized to the clock domain, it enters the glitch rejection logic. This is governed by a sequential state machine utilizing a 16-bit saturation counter.

- The logic continuously compares the incoming synchronized signal (`sync_2`) against the current registered output (`button_out`).
- If the signals differ, a transition is detected, and the 16-bit counter begins incrementing.
- **The Reset Condition:** If the input voltage spikes or drops for even a single nanosecond (representing a mechanical bounce), the counter is instantly cleared to zero.
- **The Threshold:** Only when the signal remains absolutely stable and unchanging for a predefined parameter threshold (e.g., 2,000 clock cycles) does the logic validate the physical press and update the output register.

1.1.2 Real-Time Applications

This hardware-level filtering is mandatory in physical embedded applications:

- **Industrial CNC Limit Switches:** Prevents heavy machine vibration from accidentally triggering false end-of-travel limits and halting production.
- **Human-Machine Interfaces (HMI):** Ensures membrane keypads and tactile switches register exactly one input per human interaction.
- **System Reset Controllers:** Filters noisy physical power-on-reset (POR) lines to ensure microprocessors initialize cleanly without partial reboot glitches.

1.1.3 Complete Synthesizable Verilog RTL Description

The following synthesizable RTL implements the anti-metastability synchronizer and the sequential saturation timer.

```
`timescale 1ns / 1ps

module debouncer #(
    // Parameterized stability threshold.
    // e.g., 2000 ticks at 10 MHz = 200 microseconds of required stability
    parameter DEBOUNCE_LIMIT = 16'd2000
)(
    input  wire clk,          // High-speed system clock
    input  wire rst_n,        // Active-low asynchronous reset
    input  wire button_in,    // The noisy, bouncing physical input

    output reg  button_out    // The clean, debounced digital output
);

    //------------------------------------------------------------------------
    // 1. ASYNCHRONOUS SYNCHRONIZER
    //------------------------------------------------------------------------
    reg sync_1, sync_2;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            sync_1 <= 1'b0;
            sync_2 <= 1'b0;
```

```
        end else begin
            sync_1 <= button_in;
            sync_2 <= sync_1;
        end
    end

    //--------------------------------------------------------------------------
    // 2. THE STABILITY COUNTER
    //--------------------------------------------------------------------------
    reg [15:0] counter;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            counter    <= 16'd0;
            button_out <= 1'b0;
        end else begin
            if (sync_2 == button_out) begin
                counter <= 16'd0; // Signal is stable
            end else begin
                counter <= counter + 16'd1; // Transition detected

                // Validate signal only after continuous, unbroken stability
                if (counter >= DEBOUNCE_LIMIT) begin
                    button_out <= sync_2;
                    counter    <= 16'd0;
                end
            end
        end
    end

endmodule
```

Chapter 2

Verification, Co-Simulation & Waveform Analysis

2.1 IP Block 6: Digital Debouncer Verification

2.1.1 Digital Simulation (Icarus Verilog & GTKWave)

Before analog translation, the digital accumulation logic was verified natively using a chaotic testbench in Icarus Verilog. Custom behavioral tasks were written to violently toggle the `button_in` parameter at random nanosecond intervals, simulating a cheap, bouncing mechanical spring making and breaking contact.

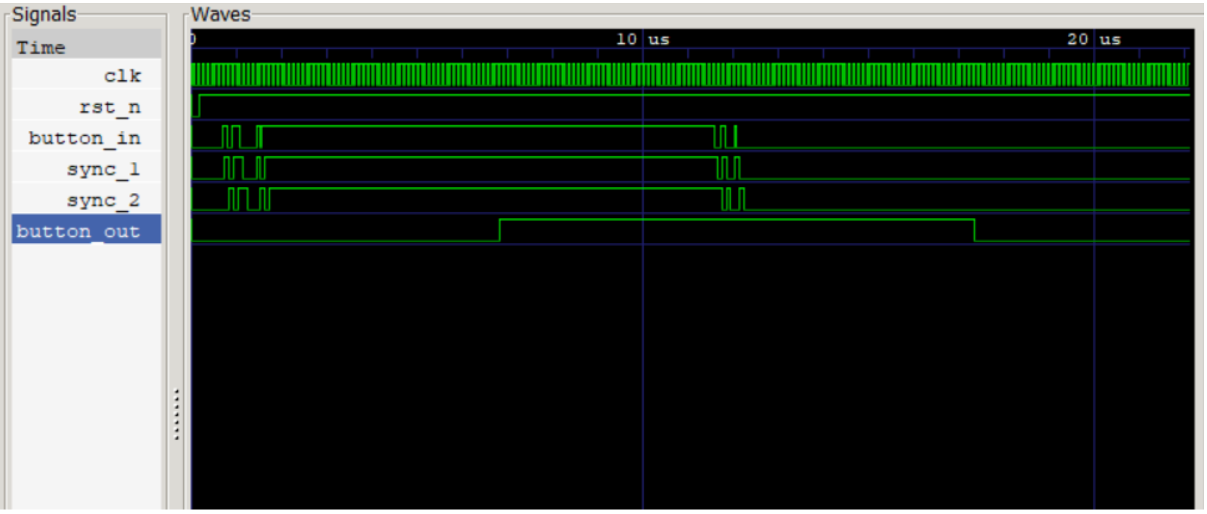

Figure 2.1: GTKWave Digital Simulation demonstrating the RTL successfully ignoring high-frequency logic toggles. The output remains clamped at 0 until the input stabilizes.

As evidenced by the digital waveform data (Figure 2.1), the `button_out` register safely ignores the rapid state changes. Only after the chaotic toggling ceases and the saturation counter strictly hits its programmed limit does the logic transition to 1.

2.1.2 System Integration (eSim Mixed-Signal Scope)

To definitively prove the IP block's resilience against physical real-world anomalies, it was subjected to an advanced analog noise simulation within the eSim NGVeri mixed-signal environment.

Schematic Wiring & PWL Noise Injection

A 10 MHz master clock and reset pins were driven by standard `vpulse` sources routed through a 2-bit `adc_bridge`.

To accurately model the physics of contact bounce, a standard square wave could not be used for the button input. Instead, a custom **Piecewise Linear (PWL)** voltage source was programmed directly into the Ngspice netlist:

```
v3 b_in gnd PWL(0 0 500u 0 500.1u 5 510u 0 530u 5 540u 0 580u 5 585u 0 600u 5)
```

This PWL array forces the analog input voltage to physically spike and crash multiple times between 0.5 ms and 0.6 ms, explicitly replicating the chaotic mechanical rattling of a raw physical switch. The filtered 1-bit digital output was routed through a `dac_bridge` for analog extraction.

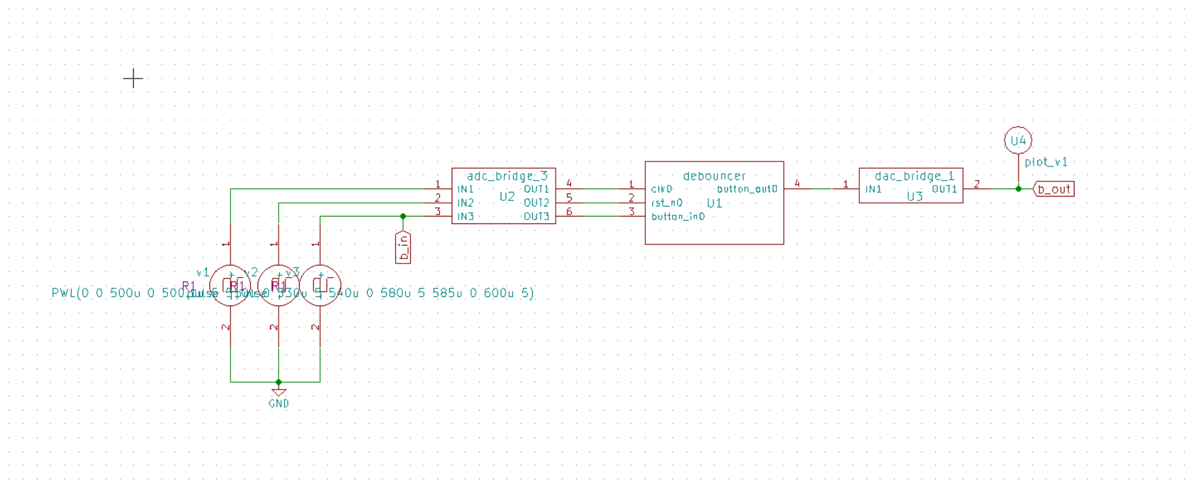

Figure 2.2: eSim Mixed-Signal Schematic illustrating the signal routing from the raw analog input, through the ADC bridge, and out to the clean DAC extraction point.

Advanced Ngspice Plot Customization

To visually prove the filtering capability, the Ngspice `.control` script was configured to vertically stack the raw, noisy analog input trace directly above the DAC-reconstructed digital output trace.

```
* Ngspice Control Block for Input/Output Comparison
.control
run
print allv > plot_data_v.txt
print alli > plot_data_i.txt

set xbrushwidth=3
set color0=black
set color1=green

* Stack the bouncing analog input (+6V offset) above the clean DAC output
plot v(b_out) (v(b_in)+6)

.endc
```

Interpretation of Output Waveforms

The transient simulation provides undeniable proof of the digital filter operating successfully across the mixed-signal boundary, converting chaos into absolute stability.

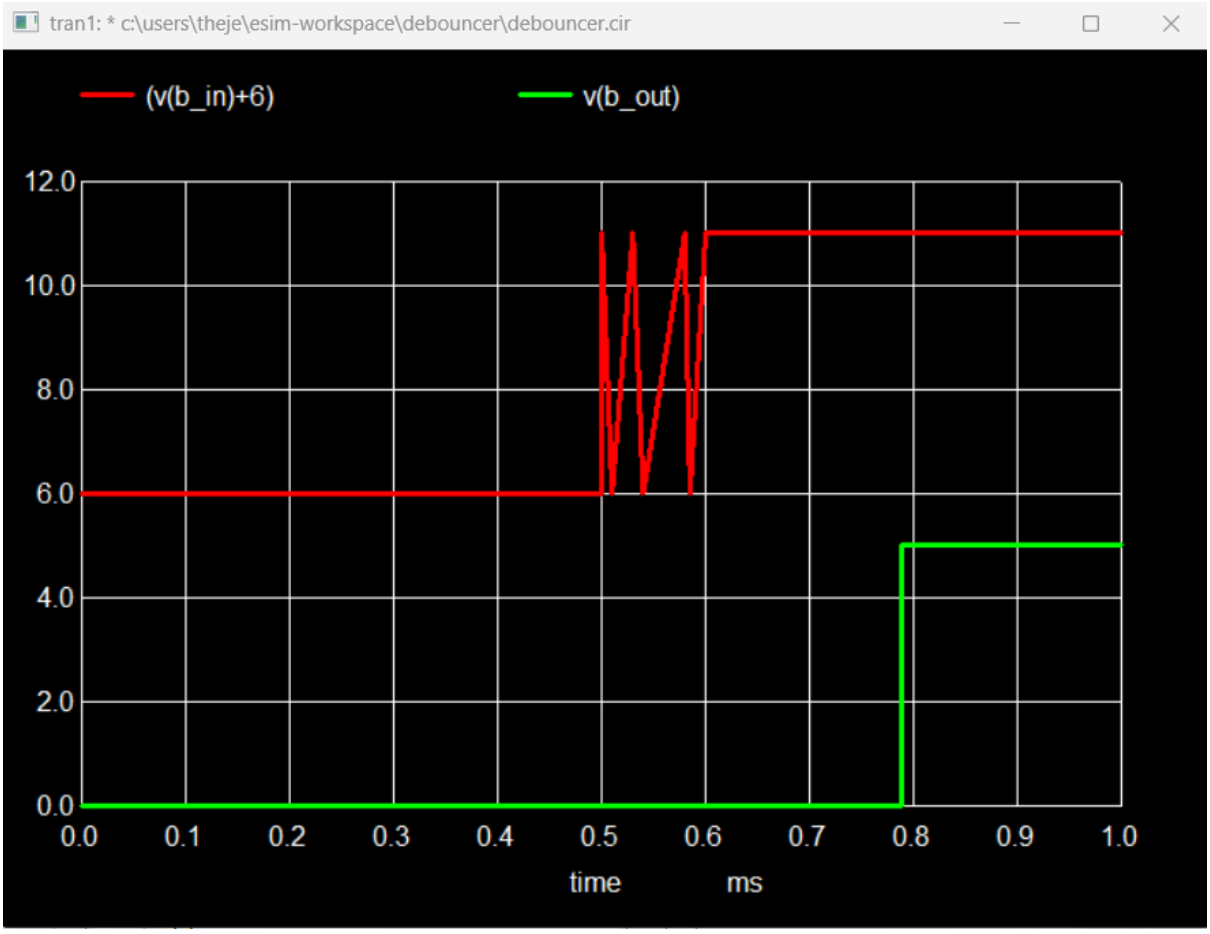

Figure 2.3: Ngspice Transient Simulation. The red trace displays the injected PWL mechanical bounce. The green trace displays the hardware delaying its output until absolute signal stability is achieved.

As evidenced by the simulation data (Figure 2.3):

1. **Absolute Glitch Rejection:** Between 0.5 ms and 0.6 ms (red trace), the analog voltage wildly oscillates between 0V and 5V. The Verilog saturation counter successfully detects these drops, continuously resetting its internal timer, and keeping the `b_out` DAC line (green trace) completely flat at 0V, protecting the downstream system from the noise.

2. **Mathematical Delay Validation:** The chaotic bouncing ceases permanently at exactly the 0.6 ms mark. The Verilog hardware parameter was configured for 2,000 clock cycles at 10 MHz, which mathematically equates to exactly 0.2 ms of real-time required stability. As predicted, the green output trace snaps cleanly to 5V at exactly the 0.8 ms boundary, validating the absolute accuracy of the co-simulation timing engine.